TURBOGATOR: TAKING A BITE OF RUNTIME FROM GATR

Adam Suma, Frederico Aberle, Jannis Alsbach, Mihai-George Licu

Department of Computer Science
ETH Zurich, Switzerland

ABSTRACT

Geometric Algebra Transformers (GATr) extend transformer architectures to structured geometric data by representing features as multivectors and using geometric-algebra-aware operations. However, the fixed tensors underlying many of these operations are highly sparse, making direct dense implementations inefficient. We present TurboGator, a specialized CPU inference implementation of the GATr forward pass. TurboGator exploits this sparsity and the regular structure of the underlying operations to reduce unnecessary computation, improve memory locality, and better utilize processor vector units. Our final implementation achieves up to a 5× speedup over the reference, while preserving the mathematical structure of the original model.

1. INTRODUCTION

The transformer architecture has revolutionized the way natural language is represented and the next word in a sequence of tokens is predicted. Because generic transformers still struggle to represent complex 3D geometric data such as points, vectors, and planes, and spatial realities such as rotations, and translations, they led to the design of more specialized transformer architectures like the Geometric Algebra Transformer. GATr addresses this by representing features as 16-dimensional multivectors in projective geometric algebra, providing a structured way to encode common geometric quantities and transformations. Compared to standard transformers, GATr replaces several ordinary vector-space operations with geometric algebra operations, allowing the model to capture structured geometric relationships between features.

The aim of this work is to accelerate inference in GATr by optimizing its core CPU kernels while preserving the model's mathematical structure.

Motivation. Computing the geometric product using standard dense tensor operations is fundamentally inefficient. For instance, a dense basis tensor allocates 4096 floating-point values per operation, even though most of these entries are zero. Furthermore, the few non-zero elements are strictly restricted to ±1. We can drastically reduce memory overhead by storing only non-zero entries, eliminate all arithmetic operations involving zero, and replace the remaining floating-point multiplications with highly efficient bitwise sign flips.

Related Work. We refer to `ezgatr` as our reference implementation. It is a Python implementation of GATr using general dense PyTorch and NumPy tensor operations and does not leverage the structure of the geometric algebra. In our own baseline implementation we directly port the reference's logic to a naive C++ implementation, where no major optimizations were performed. This implementation still relies on multi-level nested loops and large allocated basis tensors leading to extremely poor performance, operating at roughly 3% of the theoretical peak. The optimizations leverage the geometric algebra structure, consider the memory hierarchy, and fully utilize the potential of vectorization and were performed on top of this naive baseline implementation.

Contributions. We developed `TurboGator`, a highly performant implementation of the GATr. Our implementation exploits the extreme structural sparsity to reduce data size by 95%, while integrating optimizations such as tile-wise fused Flash Attention, cache blocking, and fully-pipelined 256-bit AVX vectorization. We evaluate our optimized kernels against our naive baseline and the `ezgatr` reference. Our scalar optimizations achieve up to an **11× speedup** (avg. 8.1×) over the baseline. Consequently, our final implementation (`TurboGator+`) has up to a **5× speedup** (avg. 3.8×) over `ezgatr`, establishing a highly efficient pathway for scaling GATr inference.

2. BACKGROUND ON THE ALGORITHM/APPLICATION

We summarize the GATr forward pass, introduce the kernels optimized in this work, and define the cost measures used for the performance analysis.

Description. Each GATr activation is a collection of multivectors from the geometric algebra $\mathcal{G}(3, 0, 1)$. Each multivector has $D = 16$ blade components, and each token contains C such multivector channels. All intermediate tensors therefore have shape $B \times T \times C \times D$, where B is the

batch size, T is the number of geometric objects, C is the number of channels, and $D = 16$ is the multivector dimension.

The input is first passed to `EquiLinear`, which lifts C_{in} to C_{hid} channels, then N blocks follow with a residual attention sub-block and a residual MLP sub-block each, and an output `EquiLinear` maps C_{hid} to C_{out}. An intuitive diagram of the inference pipeline can be seen in Figure 1.

`EquiLinear` is a channel-mixing map whose action on each multivector is fixed by 9 learned parameters per channel pair [1],

$$y_a = \sum_b M_{ab}\, x_b, \qquad M_{ab} = \sum_{w=1}^{9} w_{abw}\, E_w \in \mathbb{R}^{16\times 16}, \tag{1}$$

where x_b and y_a are the input and output multivectors of channels b and a, w_{abw} are the learned weights, and $E_1, \ldots, E_9$ are the fixed equivariant basis maps. The reference evaluates this as a dense 16×16 contraction. `GeometricProduct` and `EquiJoin` are bilinear products of two multivectors x and y, each a contraction with one of the constant structure tensors,

$$(x \diamond y)_i = \sum_{j,k} B_{ijk}\, x_j\, y_k, \tag{2}$$

where i, j, and k index the 16 blade components, $\diamond$ denotes either operation, and B is the corresponding $16 \times 16 \times 16$ structure tensor. `EquiGeometricAttention` is an ordinary softmax attention over the $T \times T$ object pairs,

$$\begin{aligned} A_{tt'} &= \mathrm{softmax}_{t'}\Big(\frac{1}{\sqrt{d}} \sum_c \langle q_{tc},\, k_{t'c}\rangle\Big), \\ \mathrm{out}_{tc} &= \sum_{t'} A_{tt'}\, v_{t'c}, \end{aligned} \tag{3}$$

where t and t' index the query and key objects and c the channels, q_{tc} and $k_{t'c}$ are the d_q-dimensional invariant features of the query and key multivectors ($d_q = 12$), $d = d_q C$, and $v_{t'c}$ is the value multivector. The remaining `EquiRMSNorm` and `ScalarGatedGELU` are pointwise maps. All of these operations are driven by small, highly sparse, constant tensors that are precomputed once and cached.

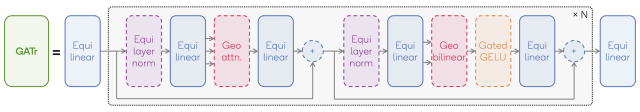

Fig. 1: Overview of the GATr architecture, from [1]. Solid boxes are learnable components, dashed boxes are fixed.

Cost measure. Initially we established a theoretical cost measure $W(n)$, the number of scalar floating-point operations of one inference forward pass for the `ezgatr` implementation. The variables are defined as previously stated, with H being the number of heads. For the channel-mixing `EquiLinear` function, C_a and C_b represent the input and output channel counts, respectively for the channel-wise kernels, C represents the single channel count. Table 1 summarizes the FLOP counts for each function.

Function	FLOP count W
`EquiLinear` $C_a \to C_b$	$2\,B\,T\,C_a\,C_b\,D^2$
`EquiGeometricAttention`	$BHT^2(56C{+}4) + 506\,BHTC$
`GeometricProduct`	$3\,B\,T\,C\,D^3$
`EquiJoin`	$3\,B\,T\,C\,D^3 + B\,T\,C\,D$
`EquiRMSNorm`	$2\,B\,T\,C\,(D{+}8) + 4\,B\,T$
`ScalarGatedGELU`	$B\,T\,C\,(D{+}4)$

Table 1: Theoretical FLOP counts per function.

Summing the work over the input and output `EquiLinear` layers and the N blocks, together with the per-token reference mean and the residual additions, gives the total $W(n)$. `EquiGeometricAttention` is the only kernel quadratic in T, so it dominates W as the sequence length increases.

While $W(n)$ characterizes the algorithmic work, we used it only as an initial reference and to quantify how much of this work our implementations eliminate relative to the dense `ezgatr` reference. Our actual cost measures are obtained from hardware performance counters via `perf`. We report performance as floating point operations per cycle, counting the retired floating-point instructions from the `FP_ARITH_INST_RETIRED` counters (scalar, 128-bit, and 256-bit operations, each weighted by their lane count) over the elapsed core cycles, and we quantify data movement as the DRAM traffic reported by the memory-controller read and write CAS counters.

3. OPTIMIZATION PERFORMED

The goal of our optimization was to accelerate the GATr forward pass by specializing the above-mentioned kernels for CPU inference.

We began by implementing a baseline C++ implementations for each of the kernels, which we aligned as best as possible with the reference implementation of ezgatr. Most notable for this step was finding the right matrix multiplication contractions to ensure a workload comparable to the torch opt-einsum operations. With our comparable baseline implemented, we developed three incremental scalar variants followed by a fully vectorized AVX2 implementation, and one final vectorized variant optimized via an exploration of compiler flags.

Throughout the process of optimizing our kernels we performed profiling which showcased an alternating bottleneck between the `EquiLinear` and `EquiGeometricAttention` kernels. Initially we focused mostly on `Equi-`

`Linear` which was dominating the runtime, but later as input sizes of our runs increased, the quadratic asymptotic complexity of the `EquiGeometricAttention` kernel took over and became the new major bottleneck.

We now present the different optimizations performed on our baseline kernels.

3.1. Loop reordering & register accumulation

The baseline `EquiLinear` kernel is structured in two passes. The first pass materializes an intermediate tensor

$$\text{kernel}[o, i, d, s] = \sum_w W[o, i, w] \, \text{basis}[w, d, s],$$

while the second pass accumulates

$$Y[b, o, d] = \sum_{i,s} \text{kernel}[o, i, d, s] \, X[b, i, s]$$

with the loop order $(b,\ o,\ d,\ i,\ s)$. Because d is the third-outermost loop, each input value $X[b, i, s]$ is loaded independently for every output blade d, yielding 16 redundant loads per (i, s) pair and preventing the compiler from keeping $X[b, i, s]$ in a register across iterations.

In our first optimization of `EquiLinear`, we address this with two coupled changes. First, the kernel is stored transposed as $\text{kernel}[o, i, s, d]$, making the d dimension stride-1 in memory. Second, the accumulation loop order is inverted to $(b,\ o,\ i,\ s,\ d)$, so d is now the *innermost* loop. A scalar register array `float acc[16]` holds all 16 blade partial sums for one (b, o) output row simultaneously. For each (i, s) step, the input scalar $x_v = X[b, i, s]$ is loaded once and multiplied against 16 consecutive kernel entries in a tight inner loop, reducing the load count by $15\times$ per (i, s) pair and enabling the compiler to keep x_v in a register for the full unrolled body.

3.2. Sparsity

All fixed geometric-algebra kernels in GATr are structurally sparse. The `GeometricProduct` basis tensor contains only 192 non-zero entries out of 4096, corresponding to a density of approximately 4.7%. The `EquiLinear` basis has 24 non-zero entries out of 2304, corresponding to approximately 1%, and the DAA (Distance-aware attention) projection tensors $B_Q, B_K \in \mathbb{R}^{4\times4\times5}$ in `EquiGeometricAttention` each contain 7 non-zero entries out of 80, corresponding to approximately 8.75%. Similarly, the `EquiJoin` kernel tensor contains 81 non-zero entries out of 4096, corresponding to a density of approximately 2%.

We represent all three tensors in COO (Coordinate format) and replace every dense loop with an enumeration over non-zero entries only. COO is appropriate here because access is always a single sequential pass per batch element and each entry list fits in L1 cache. This makes the additional overhead of formats such as CSR unnecessary.

3.3. Kernel simplification

A generic implementation of the equivariant kernels requires looping over the full basis tensor at runtime and checking or multiplying each entry. After identifying the sparse structure, a further step is to eliminate these loops entirely by inlining the non-zero contributions directly as straight-line code. This removes loop-control overhead and branch mispredictions, and exposes a fixed sequence of fused multiply-add operations that the compiler can schedule without further analysis.

For `EquiLinear`, this also eliminates the two-pass structure. Instead of first materializing an intermediate kernel tensor and then contracting it against the input, all 9 basis weights are loaded once per (o, i) channel pair, and the 24 non-zero accumulations are performed in a single unrolled block. The same simplification is applied to the DAA (Distance-aware attention) projections in `EquiGeometricAttention`, reducing an 80-operation triple loop to 5 direct assignments.

3.4. Fast exponential

The softmax in `EquiGeometricAttention` calls `std::exp` once per attention score, resulting in T^2 transcendental evaluations per head for sequence length T.

We replace `std::exp` with a custom approximation based on $e^x = 2^n e^f$, where $n = \lfloor x \log_2 e \rceil$ and $f = x - n \ln 2$. The factor 2^n is computed directly from the floating point representation exponent bits, while e^f is approximated by a degree-4 Taylor polynomial:

$$p(f) = 1 + f + \frac{f^2}{2} + \frac{f^3}{6} + \frac{f^4}{24}.$$

The input is clamped at $x = -87.34$ to avoid underflow. This replaces a full `std::exp` call with a small fixed sequence of arithmetic and integer operations. The resulting relative error of approximately 10^{-5} is acceptable because softmax normalizes the exponential values.

3.5. Online softmax

The baseline softmax over each row of the $T \times T$ attention score matrix requires three sequential passes: one to compute the row maximum, one to exponentiate and accumulate the denominator, and one to normalize. Since each pass reads the full row, the softmax-stage memory traffic scales as $3T$ per row.

We replace this with the online softmax of Milakov et al. [2], which maintains a running maximum and denomi-

nator (m_i, d_i) as each score x_i is processed:

$$m_i = \max(m_{i-1}, x_i), \qquad d_i = d_{i-1}e^{m_{i-1}-m_i} + e^{x_i-m_i}.$$

When x_i becomes the new maximum, the factor $e^{m_{i-1}-m_i}$ rescales the previous partial sum, removing the need for a separate max-finding pass. The implementation then performs a second pass to write the unnormalized exponentials and stores the reciprocal $1/d_T$ separately. This reduces the softmax row traversal from three passes to two.

We reduce the update dependency by processing four independent (m_i, d_i) lanes and merging them after the loop. The reciprocal $1/d_T$ is applied after the value contraction, so softmax divisions become a single output-row scaling. All exponentials use `fast_exp`.

3.6. Flash Attention & tiled GEMM

The baseline attention kernel materializes the full $T \times T$ score matrix before applying softmax and aggregating values. For large T, this matrix exceeds cache capacity and increases memory traffic throughout the attention computation.

For large sequence lengths, we use a Flash-Attention-style tiled implementation inspired by Dao et al. [3]. The computation is split into $B_T \times B_T$ score tiles with $B_T = 64$. Each query tile streams over key and value tiles, computes the local $QK^\top$ scores with tiled GEMM (General Matrix Multiply), updates the running softmax statistics, and immediately accumulates the corresponding value contribution. Thus, the full attention matrix is never stored; only one score tile and per-row statistics are kept at a time, reducing score working memory from $\mathcal{O}(T^2)$ to $\mathcal{O}(B_T^2)$.

Both score computation and value accumulation use a register-blocked GEMM micro-kernel with tile size $M_R \times N_C = 4 \times 4$. The accumulating variant rescales the existing output tile before adding the new contribution, fusing the Flash Attention correction step into the GEMM update.

3.7. Vectorization

All kernels were rewritten using 256-bit AVX2 intrinsics, processing 8 single-precision floats per instruction. Each kernel required a different strategy to expose data-level parallelism.

`EquiLinear` : The 16-component multivector output is split into two 8-wide AVX registers, `lo` and `hi`. The main challenge is the cross-grade coupling terms, such as basis element w_5 mapping $x[0] \rightarrow \mathrm{acc}[1]$, which require non-contiguous lanes from the input vector. We address this by pre-packing each (o, i) weight tuple into a 32-float layout aligned with the basis structure, then using `_mm256_permutevar8x32_ps` to rearrange the input lanes before each FMA. This eliminates scalar fallback paths. Packed weights are cached in a thread-local map, amortizing the packing cost across repeated forward passes. Four batch rows are processed simultaneously per output channel, keeping eight AVX accumulator registers live at once.

`EquiRMSNorm` : The equivariant norm sums the squares of only 8 of the 16 blades, corresponding to the non-null blades under the $\mathcal{G}(\mathbb{R}^{3,0,1})$ metric. Rather than extracting individual elements, we accumulate the full 16-wide squared sum into two `__m256` accumulators and then apply a compile-time constant mask via a vectorized multiply before the horizontal reduction. This selects only the contributing blades without branching. The subsequent scale-and-write pass uses 256-bit loads and stores.

`GeometricProduct` : Each multivector has 16 components, so processing one at a time leaves AVX registers at $1/8$ utilization. We instead process 8 multivectors simultaneously by transposing the input batch: blade i of all 8 multivectors is gathered into a single `__m256`, allowing one AVX FMA to advance all 8 batch elements at once. The positive and negative COO entries are stored in separate lists, mapping directly to `_mm256_fmadd_ps` and `_mm256_fnmadd_ps`, respectively. Input and output transposes surround the compute kernel to restore the original layout.

`EquiGeometricAttention` : The `fast_exp` approximation is lifted to an 8-wide AVX2 variant, `fast_exp_ps`, with all scalar operations replaced by their `__m256` equivalents, including integer exponent construction via `_mm256_slli_epi32`. The online softmax and Flash Attention inner loops then operate on vectors of 8 scores at a time using this vectorized exponential.

`EquiJoin` and `ScalarGatedGELU` follow simpler patterns. For `EquiJoin`, the COO entries are reorganized by output blade i so that up to 8 (j, k) pairs per blade can be gathered simultaneously with `_mm256_i32gather_ps` and accumulated via FMA. For `ScalarGatedGELU`, the gate is computed from a single scalar blade and broadcast into an AVX register, then applied to the full 16-component multivector with two 256-bit multiplies.

3.8. Binding Optimizations "The Glue"

While the kernel optimizations above provided the bulk of our speedup, what ultimately let us surpass `ezgatr` were improvements at the C++/Python boundary, where each pybind11 call can incur hidden overhead. While we recognise this is not the main focus area of the course we have considered the work here particularly interesting. PyTorch inserts `.contiguous()` copies when a kernel receives a strided view, and chaining kernels requires reassembling intermediate results with operations such as `torch.cat`. As such the kernels were made stride aware, operating directly on the strided tensors passed from Python so that no contiguous copies are materialized. In the bilinear block, where the

`GeometricProduct` and `EquiJoin` results were previously concatenated, we instead preallocate a single output buffer and pass two slices of it to the kernels that write in place. A single contiguous tensor is then forwarded to the output projection, eliminating both the per kernel copies and the concatenation.

3.9. Experimental Compiler Flags

As a final attempt to extract additional performance without further changes to the kernels, we experimented with a set of aggressive compiler optimization flags, producing the variant we refer to as `TurboGator+`. Namely:

```
-fprefetch-loop-arrays
-funroll-loops
-ffast-math
-ffinite-math-only
-fno-trapping-math
-flto=auto
-fno-semantic-interposition
-mprefer-vector-width=256
```

Cumulatively these flags improved both runtime and performance by approximately 10% over `TurboGator`. The bulk of this gain came from combining `-fprefetch-loop-arrays` and `-funroll-loops`. The remaining flags each contributed below 1% individually, but were kept in the final configuration.

3.10. Gator Models

We now present which optimizations are included in each model. The variants are incremental. All ScalarGator variants are purely scalar implementations: they contain no hand-written SIMD intrinsics, and autovectorization is disabled using `-fno-tree-vectorize`. Thus, the comparison between ScalarGatorV3 and TurboGator isolates the effect of explicit hand-written AVX2 vectorization. When compiled with auto-vectorization enabled, the compiler vectorizations did not have any further impact on the performance of TurboGator.

Variant	Additions over previous
ScalarGatorV1	Loop reordering, register accumulation, sparsity and kernel simplification except in `EquiLinear`.
ScalarGatorV2	Sparsity and kernel simplification in `EquiLinear`.
ScalarGatorV3	Fast exponential, online softmax, Flash Attention, and measured tile sweeps.
TurboGator	AVX2 intrinsics, binding optimizations.
TurboGator+	Experimental compiler flags.

4. EXPERIMENTAL RESULTS

We evaluate the optimized implementations in terms of runtime, work reduction, hardware utilization, and memory behavior across increasing input sizes.

4.1. Experimental setup

All measurements are taken on a single performance core of an Intel Core 7 240H, a Raptor Lake CPU on the Raptor Cove (derived from Golden Cove) microarchitecture [4]. We disable turbo boost and pin the benchmark to a single core (core 4) running at a fixed $2.5\,\text{GHz}$ with $32\,\text{GB}$ of LPDDR5 memory. We completely disable its logical sibling (core 5). The machine supports SSE4.1/4.2, AVX2, and FMA3. A core has access to a $48\,\text{KiB}$ L1d, a $1.25\,\text{MB}$ L2, and a shared $24\,\text{MB}$ L3. The machine's theoretical scalar peak is $\pi_s = 5$ and the vectorized one is $\pi_v = 40$ FLOPs/cycle. These uneven throughput values reflect the design featuring two FMA ports and one dedicated addition port. The memory bandwidths are $\beta = 96, 64, 32$, and 9 bytes/cycle to L1, L2, L3, and DRAM respectively.

Compilation was done with `g++ (Debian 14.2.0-19) 14.2.0` under the C++17 standard. The common baseline flags are: `-O3 -march=native -fipa-pta -falign-loops=32 -fno-math-errno -fno-tree-vectorize -fno-tree-slp-vectorize`

The additional aggressive flags for `TurboGator+` are listed in Section 3.9.

The input is a batch of $B = 8$ random multivectors of shape $B \times T \times C_{\text{in}} \times D$ ($D = 16$) run through the full forward pass. A single parameter N sets $T = 32N$ and $C_{\text{in}} = 2N$; we sweep $N \in \{1, 4, 6, 16, 32, 48, 56, 64\}$ to cross all cache regimes and make the largest sizes run for seconds to minutes.

Every build is first validated against `ezgatr` with identical weights. Timing pins the process with `taskset`, runs single threaded, discards warm-up passes, and reports the median of the timed passes. The `perf` recorded counters are gated to the timed passes only and give cycles, instructions, retired scalar/128/256-bit FP ops (Work), L1/L2 and L2/L3 line traffic, and DRAM CAS counts (Data Movement). For stability we accept runs only if the coefficient of variation (CV) and standard deviation remain below 1%. We initially ran 10 warm-up and 40 timed iterations, but the CV settled well within this bound after only a few iterations (typically $<0.3\%$), so we kept a safe 5 warm-up and 9 timed runs configuration. Hot spots were profiled separately with `py-spy` and the PyTorch profiler.

4.2. Results

Attention AV tiling. To pick the register-tile shape of the attention AV product, we ran a microbenchmark on the server

that reproduces our exact kernel. Figure 2 shows the sweeps $M_R \times N_R$. Small tiles expose too few FMAs to make up for latency, large ones spill the 16 `YMM` registers. In our case 4×2 is optimal and T is always divisible by 4.

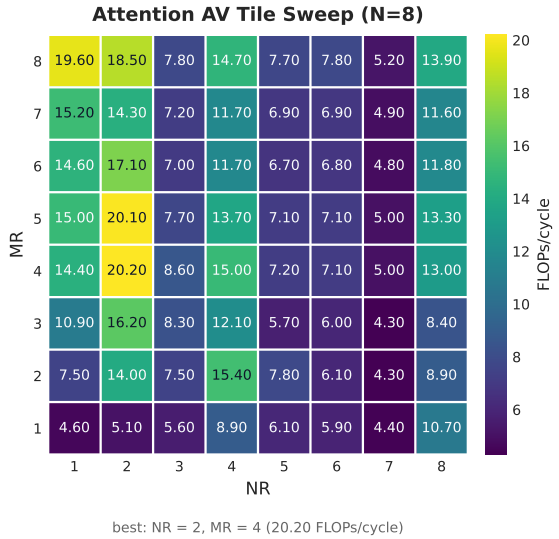

Fig. 2: Attention locality tile sweep.

Work efficiency. Figure 3 reports work efficiency, the percentage of the dense analytic work $W(n)$ actually executed. The baseline and `ScalarGatorV1` stay near 90%, while sparsity and kernel simplification cut `ScalarGatorV2` to ∼28% on average. Its curve still climbs with N and overtakes the later variants (45% at N=64) because it calls `std::exp` once per attention score ($\Theta(T^2)$ calls). The function retires far more floating point instructions than the single FLOP the analytic model assigns an exponential, inflating its measured work as attention comes to dominate. `ScalarGatorV3`'s `fast_exp` polynomial removes this overhead, keeping its executed work low and roughly flat.

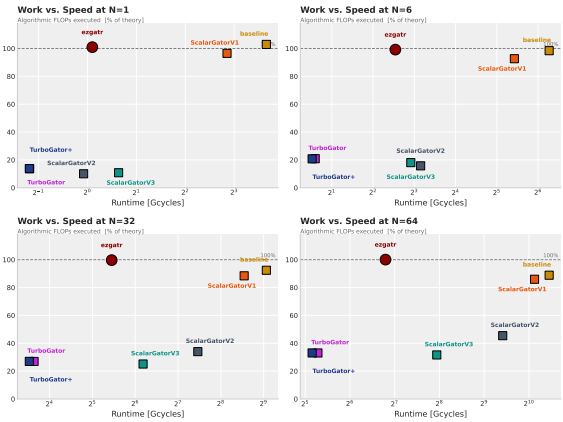

Fig. 3: Work efficiency.

Performance analysis. Figures 4 and 5 give FLOPs/cycle per size for the dense and sparse versions respectively. `TurboGator+` reaches 18.7 FLOPs/cycle (≈47% of $\pi_v = 40$), matching dense `ezgatr` (16.9) while doing ∼4× less work. The scalar versions reach only ∼2.4 (∼50% of $\pi_s = 5$ but ∼6% of π_v). The AVX2 addition lifts throughput from 2.4 to 17.3 FLOPs/cycle, a ∼7× gain that recovers most of the 8× ceiling of 256-bit single-precision SIMD. The dotted vertical lines mark the input size at which the working set outgrows the 1.25 MB L2 and 24 MB L3 caches (at $N \approx 12$ and $N \approx 57$); beyond each line, accesses spill to the next, slower level. Performance climbs while the working set stays in cache and is barely affected at the L2 line, as the kernels stay compute bound, rolling off only past the L3 line as traffic reaches DRAM (the drop at $N = 64$). No L1d line is present because the reused operands, `EquiLinear`'s weight panels and the per-tile $B \times C$ multivectors, already exceed the 48 KiB L1d even at N=1, so the kernels are L2-resident at best.

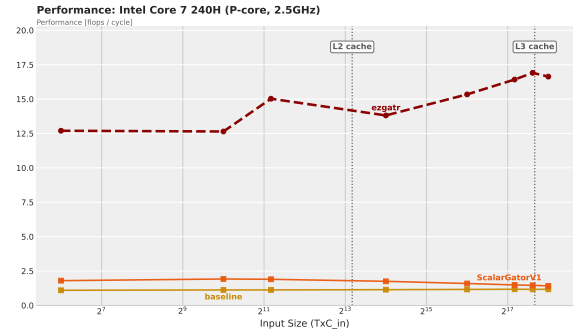

Fig. 4: Performance (FLOPs/cycle) across N, high op-count versions.

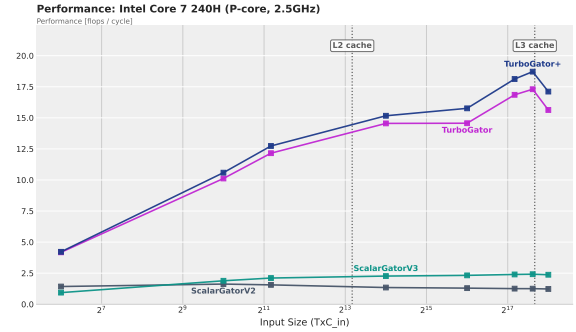

Fig. 5: Performance (FLOPs/cycle) across N, low op-count versions.

Runtime across input sizes. Since our sparse kernels execute far fewer operations than the dense reference, runtime is the only directly comparable metric. It grows quadratically in T, dominated by $\Theta(T^2)$ attention. The naive baseline is over an order of magnitude slower and is dropped for legibility in Fig. 6; Figure 7 isolates `ezgatr` against our vectorized variants.

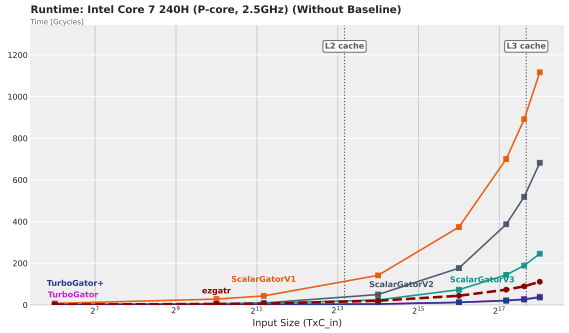

Fig. 6: Total runtime across input size N; all variants (baseline omitted).

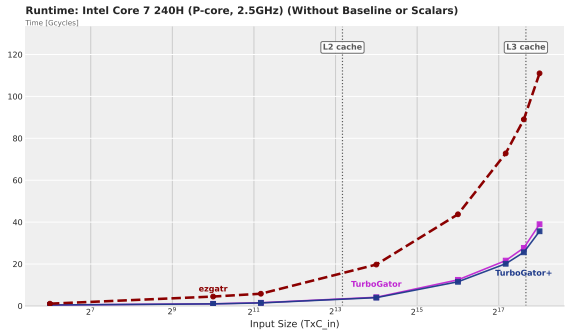

Fig. 7: Total runtime across input size N; `ezgatr` vs. vectorized variants.

Roofline analysis. Figures 8 and 9 place the dense and sparse versions on a DRAM roofline. Operational intensity is calculated with DRAM traffic, measured warm with warmups discarded. Almost all points sit right of the ridge (≈ 4.4 FLOPs/byte), so the kernels are compute bound. The low intensity at small N is due to overhead.

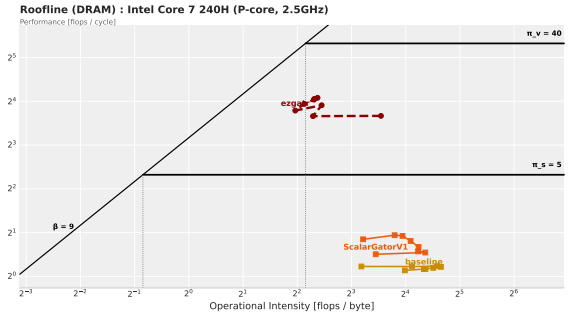

Fig. 8: DRAM roofline, high-op-count (dense) versions.

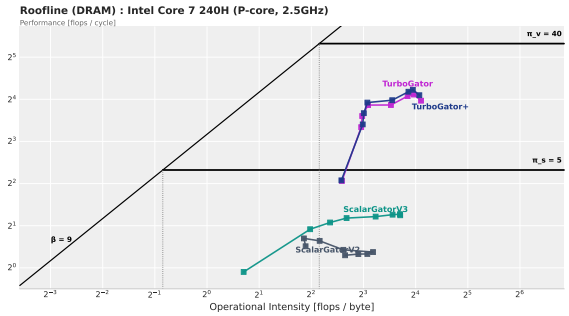

Fig. 9: DRAM roofline, low-op-count (sparse) versions.

Speedup. We break down the performance gains over `ezgatr` in two ways. Figure 10 is the *runtime* speedup (cycles): `TurboGator+` is 2.4–5.0× faster (avg. 3.8×), narrowing at large N as both saturate the same attention traffic. Figure 11 is the *performance* ratio (FLOPs/cycle), where `TurboGator+` shows a slight improvement (≈1.1×). Overall the runtime win comes largely from executing far less work, not from the marginal performance increase. Against the naive baseline the gains are far larger: `TurboGator+` is up to 58× faster in runtime and 16× higher in performance.

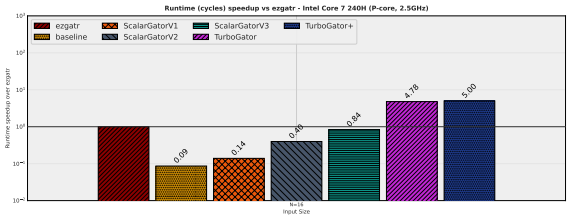

Fig. 10: **Runtime** speedup over `ezgatr` (cycles).

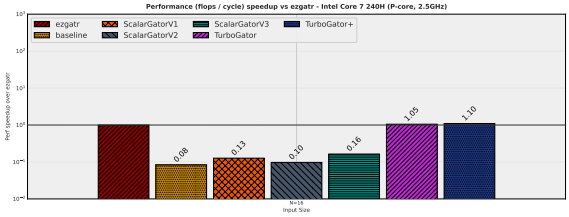

Fig. 11: **Performance** ratio over `ezgatr` (FLOPs/cycle;).

5. CONCLUSIONS

TurboGator shows that GATr inference can be accelerated by exploiting the sparsity and regular structure of its geometric-algebra kernels. By removing dense intermediate contractions, improving locality, and vectorizing the main kernels, we reduce the work performed while preserving the original model and achieving a speedup of up to 5× compared to `ezgatr`. Moreover, TurboGator+ also achieves comparable FLOPs-per-cycle performance to `ezgatr`, while also performing less arithmetic overall. Together, these improvements make GATr inference both faster and more resource-efficient, improving its scalability to larger geometric inputs.

6. REFERENCES

[1] J. Brehmer, P. De Haan, S. Behrends, and T. S. Cohen, "Geometric algebra transformer," in *Advances in Neural Information Processing Systems*, 2023, vol. 36, pp. 35472–35496.

[2] Maxim Milakov and Natalia Gimelshein, "Online normalizer calculation for softmax," arXiv:1805.02867, 2018.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," arXiv:2205.14135, 2022.

[4] Chester Lam, "Popping the hood on Golden Cove," https://chipsandcheese.com/p/popping-the-hood-on-golden-cove, 2021, Chips and Cheese. Accessed: 2026-06-20.

7. CONTRIBUTIONS OF TEAM MEMBERS

Adam Suma. Baseline and first scalar optimization performed on `equi_geometric_attention` (`opt_v1`: Sparsity, Simplicity and basic cache blocking for better locality). Performed sparsity optimization for `equi_geometric_product` (`opt_v1`). Implemented the vectorization of `equi_linear`.

Frederico Aberle. Baseline implementation of `scalar_gated_gelu`. Implementation of sparse kernel basis and hardcoding the output assignments for `equi_join`. Vectorization of `equi_join` and `rms_norm`.

Jannis Alsbach. Baseline implementation and scalar optimizations for `equi_linear`: Loop reordering, register accumulation, sparsity exploitation, loop unrolling. Implemented the vectorization of `equi_geometric_product` and `scalar_gated_gelu`.

Mihai-George Licu. Baseline implementation of `geometric_product`. Further optimization of `equi_geometric_attention`: online softmax, `fast_exp`, and FlashAttention. Binding optimizations. Compiler flags exploration (`TurboGator+`). Microarchitecture details, cost analysis and roofline modeling.

8. CODE

The most performance critical part of our fastest scalar code. Particularly `equi_linear_opt_v2.cpp`

```
equi_linear_opt_v2.cpp:49-116
for (size_t b = 0; b < batch; ++b) {
    for (size_t oc = 0; oc < out_channels; ++oc) {
        float acc[16] = {0.0f};

        for (size_t ic = 0; ic < in_channels; ++ic) {
            const float* w_oi =
                w_use.data() + (oc * in_channels + ic) * 9;
            const float* x_bi =
                x + (b * in_channels + ic) * 16;

            // load weights for (oc, ic)
            const float w0 = w_oi[0];
            const float w1 = w_oi[1];
            const float w2 = w_oi[2];
            const float w3 = w_oi[3];
            const float w4 = w_oi[4];
            const float w5 = w_oi[5];
            const float w6 = w_oi[6];
            const float w7 = w_oi[7];
            const float w8 = w_oi[8];

            // w=0
            acc[0] += w0 * x_bi[0];

            // w=1
            acc[1] += w1 * x_bi[1];
            acc[2] += w1 * x_bi[2];
            acc[3] += w1 * x_bi[3];
            acc[4] += w1 * x_bi[4];

            // w=2
            acc[5] += w2 * x_bi[5];
            acc[6] += w2 * x_bi[6];
            acc[7] += w2 * x_bi[7];
            acc[8] += w2 * x_bi[8];
            acc[9] += w2 * x_bi[9];
            acc[10] += w2 * x_bi[10];

            // w=3
            acc[11] += w3 * x_bi[11];
            acc[12] += w3 * x_bi[12];
            acc[13] += w3 * x_bi[13];
            acc[14] += w3 * x_bi[14];

            // w=4
            acc[15] += w4 * x_bi[15];

            // w=5
            acc[1] += w5 * x_bi[0];

            // w=6
            acc[5] += w6 * x_bi[2];
            acc[6] += w6 * x_bi[3];
            acc[7] += w6 * x_bi[4];

            // w=7
            acc[11] += w7 * x_bi[8];
            acc[12] += w7 * x_bi[9];
```

```
            acc[13] += w7 * x_bi[10];

            // w=8
            acc[15] += w8 * x_bi[14];
        }

        float* out_bo =
            out + (b * out_channels + oc) * 16;
        for (size_t d = 0; d < 16; ++d)
            out_bo[d] = acc[d];
    }
}
```

The most performance critical part of our fastest SIMD code. Particularly `equi_linear_vectorized.cpp`

```
equi_linear_vectorized:43-60;109-152
__attribute__((always_inline))
static inline void fma_row(__m256& acc_lo,
                           __m256& acc_hi,
                           __m256 w0,
                           __m256 w1,
                           __m256 w2,
                           __m256 w3,
                           const float* __restrict__ xp,
                           __m256i perm_lo,
                           __m256i perm_hi) {
__m256 xlo = _mm256_loadu_ps(xp);
__m256 xhi = _mm256_loadu_ps(xp + 8);

acc_lo = _mm256_fmadd_ps(w0, xlo, acc_lo);
acc_hi = _mm256_fmadd_ps(w1, xhi, acc_hi);

acc_lo = _mm256_fmadd_ps(
    w2, _mm256_permutevar8x32_ps(xlo, perm_lo), acc_lo);
acc_hi = _mm256_fmadd_ps(
    w3, _mm256_permutevar8x32_ps(xhi, perm_hi), acc_hi);
}
...

for (size_t oc0 = 0; oc0 < out_channels; oc0 += NC) {
    const size_t oc_end = std::min(oc0 + NC, out_channels);

    size_t b0 = 0;
    for (; b0 + 4 <= batch; b0 += 4) {
        const float* xr0 = x + (b0 + 0) * x_stride;
        const float* xr1 = x + (b0 + 1) * x_stride;
        const float* xr2 = x + (b0 + 2) * x_stride;
        const float* xr3 = x + (b0 + 3) * x_stride;

        for (size_t oc = oc0; oc < oc_end; ++oc) {
            __m256 lo0 = _mm256_setzero_ps();
            __m256 hi0 = _mm256_setzero_ps();
            __m256 lo1 = _mm256_setzero_ps();
            __m256 hi1 = _mm256_setzero_ps();
            __m256 lo2 = _mm256_setzero_ps();
            __m256 hi2 = _mm256_setzero_ps();
            __m256 lo3 = _mm256_setzero_ps();
            __m256 hi3 = _mm256_setzero_ps();

            const float* wp =
                w_packed + oc * in_channels * PACK;
            for (size_t ic = 0; ic < in_channels; ++ic) {
                const float* w = wp + ic * PACK;
                __m256 w0      = _mm256_loadu_ps(w);
                __m256 w1      = _mm256_loadu_ps(w + 8);
                __m256 w2      = _mm256_loadu_ps(w + 16);
                __m256 w3      = _mm256_loadu_ps(w + 24);
                const size_t o = ic * 16;
                fma_row(lo0, hi0, w0, w1, w2, w3,
                        xr0 + o, perm_lo, perm_hi);
                fma_row(lo1, hi1, w0, w1, w2, w3,
                        xr1 + o, perm_lo, perm_hi);
                fma_row(lo2, hi2, w0, w1, w2, w3,
                        xr2 + o, perm_lo, perm_hi);
                fma_row(lo3, hi3, w0, w1, w2, w3,
                        xr3 + o, perm_lo, perm_hi);
            }

            float* o0 =
                out + ((b0 + 0) * out_channels + oc) * 16;
            float* o1 =
                out + ((b0 + 1) * out_channels + oc) * 16;
            float* o2 =
                out + ((b0 + 2) * out_channels + oc) * 16;
            float* o3 =
                out + ((b0 + 3) * out_channels + oc) * 16;
            _mm256_storeu_ps(o0, lo0);
            _mm256_storeu_ps(o0 + 8, hi0);
            _mm256_storeu_ps(o1, lo1);
            _mm256_storeu_ps(o1 + 8, hi1);
            _mm256_storeu_ps(o2, lo2);
            _mm256_storeu_ps(o2 + 8, hi2);
            _mm256_storeu_ps(o3, lo3);
            _mm256_storeu_ps(o3 + 8, hi3);
        }
    }
```